

Gram Aarogya Seva

Comprehensive User, Feature & File Access Guide

Project: Gram Aarogya Seva (UBA Healthcare)	Platform: Flutter (Android-first) + Firebase
Backend Region: asia-south1 (Mumbai)	Root Location: D:\gram_aarogya_seva-main
Entry File: lib/main.dart	Date Generated: September 2026

1. Executive Overview & System Purpose

Gram Aarogya Seva is a specialized rural healthcare appointment and clinic management platform built for the **Unnat Bharat Abhiyan (UBA)** initiative. It bridges the gap between rural villagers and verified medical professionals by providing real-time doctor availability schedules, appointment booking, and digital health summaries. The application is tailored for low-end mobile devices operating on low-bandwidth rural networks, with multi-lingual support in **Hindi, Marathi, and English**.

2. How to Access & Run the Project

The codebase is fully contained inside **D:\gram_aarogya_seva-main**.

Step 1: Open Terminal in Project Root

Navigate to the main directory using PowerShell or Command Prompt:

```
cd D:\gram_aarogya_seva-main
```

Step 2: Connect a Device or Emulator

Ensure an Android smartphone with USB Debugging enabled is connected via USB, or launch an Android emulator. Verify with:

```
flutter devices
```

Step 3: Run / Install the Application

Execute any of the following standard Flutter commands:

Command	Purpose & Description
flutter run	Builds and installs the app on your connected device in Debug/Live mode.
flutter install	Installs a previously built APK onto the connected Android device.
flutter build apk --release	Compiles a production-ready release APK file (saved in build/app/outputs/flutter-apk/app-release.apk).
flutter analyze	Runs static analysis to check Dart code health and syntax rules.
flutter test	Executes Dart unit tests for providers, validators, and date logic.

3. Roles & Functional Features

Gram Aarogya Seva supports four distinct user roles, each guarded by role-based routing (GoRouter):

Role	Key Features & Capabilities	Entry Point Route
Patient (Villager)	• Phone OTP login & profile management • Discover verified doctors filtered by village • 3-step appointment booking wizard • Appointment cancellation (≥ 2h before slot) • Read doctor visit summaries & prescriptions	/patient/dashboard
Doctor	• ABDM Aadhaar OTP self-registration • Health centre selection & availability manager • Accept, reject, or complete appointments • Record patient visit notes & follow-up recommendations	/doctor/dashboard
Operator (Healthcare Worker)	• Register villagers who do not own smartphones • Book appointments on behalf of walk-in patients • Manage community queue at health centers	/operator/dashboard
Admin (System Admin)	• Review and approve/reject doctor registrations • Add and manage Villages & Primary Health Centres (PHCs) • Monitor platform usage and assign system roles	/admin/dashboard

4. Project File Structure & Access Map

Below is a detailed map of where specific features and source files live inside the project:

File / Folder Path	Responsibility & Contents
lib/main.dart	App entryptoint. Initializes Firebase, Riverpod containers, offline persistence, and locale settings.
lib/app.dart	Root widget containing MaterialApp.router, ThemeData, and localization delegates.
pubspec.yaml	Project manifesto declaring dependencies (Riverpod, GoRouter, Freezed, Firebase SDKs) and assets.
lib/core/config/app_constants.dart	Defines database collection names, role string keys, status enums, and app routes.
lib/core/router/app_router.dart	GoRouter setup with role guards to ensure patients, doctors, and admins remain in their scope.
lib/core/services/firestore_service.dart	Read layer for client queries (fetching profiles, streaming slots, appointment history).
lib/core/services/functions_service.dart	Write layer calling Cloud Functions (atomic booking, cancellations, status transitions).
lib/features/patient/	Patient screens: dashboard, profile editor, booking wizard (3 steps), health records.
lib/features/doctor/	Doctor screens: ABDM registration, schedule slot manager, patient consultation queues.

<code>lib/features/admin/</code>	Admin screens: doctor verification queue, village/PHC configuration, platform metrics.
<code>lib/features/operator/</code>	Operator screens: walk-in patient registration and proxy appointment booking.
<code>assets/l10n/</code>	Localization translation files in JSON format (en.json, hi.json, mr.json).
<code>functions/src/</code>	Backend TypeScript Cloud Functions enforcing multi-document atomic operations and security.
<code>firestore.rules</code>	Database security rules denying illegal direct client writes and guaranteeing role isolation.

5. Technical Architecture & Security Model

Gram Aarogya Seva follows a **Server-Authoritative Write Model** to prevent data corruption and enforce strict business rules:

- **Single Document Reads:** Executed safely via FirestoreService with local offline caching.
- **Multi-Document Operations:** All multi-document mutations (booking an appointment, cancelling a slot, doctor registration, setting slot availability) must go through FunctionsService to Cloud Functions. Direct client writes for these operations are explicitly blocked by firestore.rules.
- **Aadhaar & ABDM Verification:** Doctor Aadhaar numbers are hashed server-side using a secret HMAC key; raw Aadhaar values are never stored in client state or plain database fields.

6. Summary Checklist for Quick Setup

- **Primary Folder:** D:\gram_aarogya_seva-main
- **Main App Launcher:** lib/main.dart
- **Quick Run Command:** flutter run
- **Release Build Command:** flutter build apk --release
- **Git Repository Branch:** main (local repository initialized with initial commit)